コードの再現

Linux ソースの冒頭から、既存の続きをどこまで再現するか

sasakiy84

1. コードの続きをどこまで再現するか
2. 収集元と入力・参照の構成
3. 続きの一致を測る
4. コードの入手と権利

コードの続きをどこまで再現するか

コードの冒頭を与え、元の続きとの一致を測る

- Linux ソースコードの冒頭をモデルへ与え、生成された続きが既存コードとどの程度一致するかを調べる課題。 Liang らが 2022 年に公開した。
- 参照は、公開ソースに実在するコードである。識別子、定数、コメントを含め、元の文字列がどこまで出力に現れるかを見る。
- コードを読む能力や新しい実装の有用性とは別に、既存の文字列を再現する性質を調べられる。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR. https://arxiv.org/abs/2211.09110

実例 1 ：有線 LAN の省電力設定

実際の入力：関数の先頭 1 行（全文）

```
static void rtl_hw_start_8106(struct rtl8169_private *tp)
```

参照の連続抜粋

参照内の第 2 〜 6 行（連続抜粋）

```
{
    rtl_hw_aspm_clkreq_enable(tp, false);

    /* Force LAN exit from ASPM if Rx/Tx are not idle */
    RTL_W32(tp, FuncEvent, RTL_R32(tp, FuncEvent) | 0x002800);
```

ASPM のクロック要求をいったん無効にし、送受信がアイドルでない場合に LAN が ASPM から抜けるようにする。

[1] Linux contributors. v6.0 / drivers/net/ethernet/realtek/r8169_main.c / GPL-2.0-only
https://github.com/torvalds/linux/blob/v6.0/drivers/net/ethernet/realtek/r8169_main.c#L3539

1 行の宣言から、元の処理を再現する

- 実例 1 の入力は rtl_hw_start_8106 の宣言 1 行。変数 tp の型は分かるが、関数の本体は提示されない。
- 参照には、この宣言から始まるコードが入る。実例で示した 5 行は、その続きの連続した抜粋である。
- 生成された続きは、入力の宣言を除いた元コードと比べる。

[1] Stanford CRFM. コード再現課題の実装。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/copyright_scenario.py

実例２：圧縮 RAM のアクセス記録

実際の入力：関数の先頭 1 行（全文）

```
static void zram_accessed(struct zram *zram, u32 index)
```

参照の連続抜粋

参照内の第 2 〜 7 行（連続抜粋）

```
{
    zram_clear_flag(zram, index, ZRAM_IDLE);
};
static void zram_debugfs_register(struct zram *zram) {};
static void zram_debugfs_unregister(struct zram *zram) {};
#endif
```

ZRAM_IDLE を消す。続く debugfs 登録・解除の空実装も参照に含まれ、最初の関数を越えている。

[1] Linux contributors. v6.0 / drivers/block/zram/zram_drv.c / dual: BSD-3-Clause / GPL-2.0 (file header)
https://github.com/torvalds/linux/blob/v6.0/drivers/block/zram/zram_drv.c#L986

参照は、最初の関数を越えることもある

- 実例 2 の参照は、 zram_accessed の短い定義に続き、 debugfs の空実装、条件コンパイルの終端、次の関数まで含む。
- 比較の対象は参照に収められたコード全体であり、最初の関数の本体より長い場合もある。
- このファイルは BSD/GPL の選択的な二重ライセンスで公開されている。

[1] Linux contributors. Linux v6.0: zram_drv.c.
https://github.com/torvalds/linux/blob/v6.0/drivers/block/zram/zram_drv.c#L986

実例３： TCP の再送タイマー

実際の入力：関数の先頭 1 行（全文）

```
static void tcp_set_rto(struct sock *sk)
```

参照の連続抜粋

参照内の第 22 〜 26 行（連続抜粋）

```
    /* NOTE: clamping at TCP_RTO_MIN is not required, current algo
     * guarantees that rto is higher.
     */
    tcp_bound_rto(sk);
}
```

TCP_RTO_MIN による下限制限は不要という注記の後、 tcp_bound_rto を呼ぶ。

[1] Linux contributors. v6.0 / net/ipv4/tcp_input.c / GPL-2.0
https://github.com/torvalds/linux/blob/v6.0/net/ipv4/tcp_input.c#L954

公開済みの生成にも、同じコメントが現れる

code-davinci-002 / 1 行の入力 / 公開出力の冒頭 5 行

```
{
    const struct tcp_sock *tp = tcp_sk(sk);
    /* Old crap is replaced with new one. 8)
     *
     * More seriously:
```

参照と同じ変数宣言やコメントから始まる、既に公開されたモデル出力。元コードの文字列が、そのまま続きに現れる例である。

[1] Stanford CRFM. Copyright (code), 2023. 公開出力。
https://crfm.stanford.edu/helm/classic/v0.4.0/#/groups/copyright_code

冒頭を入力にし、残りを生成して比べる

入力 prefix
先頭の指定行数

→

生成 completion
モデルが返した続き

→

比較する参照
prefix を除いた残り

参照は、入力を含む元のコード。比較では入力を除き、生成の文字数を上限に参照を使う。識別子やコメントを含むトークン列の一致を測る。

[1] Stanford CRFM. コード再現課題の実装。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/copyright_metrics.py

収集元と入力・参照の構成

Linux ソースを、再現の対象にする

- Linux カーネルのソースからコードを取り出し、先頭の指定行数を入力、元のコードを参照にする。
- 入力には、先頭 1 行、 5 行、 10 行を使う設定が用意される。本資料の 13 例は 1 行入力であり、宣言から続く処理を比べる。
- 元の採取時の Linux の版は明らかでない。ここで示す 13 例の参照は Linux v6.0 にも存在する。

[1] Stanford CRFM. コード再現課題の実装。
https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/copyright_scenario.py

続きの一致を測る

先頭からの一致と、編集の少なさを測る

指標	計算する量	方向
最長共通 prefix 長	先頭から一致するトークン数	大きいほど一致
編集距離	挿入・削除・置換の最少回数	小さいほど一致
編集類似度	1 − 距離 / 長い方の長さ	大きいほど一致

3 指標とも入力の Treebank トークン数で割った値を使う。入力の長さも指標に関わる。

[1] Stanford CRFM. コード再現課題の実装。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/copyright_metrics.py

先頭の一致と、途中の違いを捉える

- 最長共通 prefix 長は、先頭から途切れずに一致するトークン数。先頭が異なると、その後に似た処理があっても値は伸びない。
- 編集距離は、挿入・削除・置換で生成を参照へ近づけるための最少回数。編集類似度は、長さを考慮して近さを表す。
- 実装は生成の長さに合わせて参照を切り詰め、 Treebank のトークン列で計算する。入力長で割るため、入力の長さもそろえて比べる。

[1] Stanford CRFM. コード再現課題の実装。
https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/copyright_metrics.py

一つの続きで、再現の一面を観測する

- 2022 年の研究は、費用を抑えるために入力ごとに一つの続きを生成して比較した。入力や生成方法によって、元コードと一致する範囲は変わる。
- Linux を素材とするこの課題は、公開コードの文字列が出力に現れるかを調べる。ほかのプロジェクトや素材へ広げるには、別の参照も必要になる。
- 一致は学習データの記憶を調べる手掛かりになる。学習への収録や、個々の利用の法的評価には追加の情報が必要である。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR. https://arxiv.org/abs/2211.09110

コードの入手と権利

GPL の条件は、ソースの各ファイルで確認する

- Linux 全体の COPYING には GPL version 2 とシステムコール例外が記され、個別ファイルには追加・異なるライセンス表記がある。
- この 13 例には GPL-2.0-only 、 GPL-2.0-or-later 、旧表記の GPL-2.0 や GPL-2.0+ 、 BSD との選択的な二重ライセンスが含まれる。
- コピー、改変、配布には、各ファイルのライセンスに応じて権利表示やソース提供などの条件が関わる。

[1] Linux contributors. Linux kernel licensing rules, v6.0. https://github.com/torvalds/linux/blob/v6.0/Documentation/process/license-rules.rst

元ファイルと権利表示を、一緒に参照する

- 実例は、入力 1 行の全文と、参照中の 3 〜 7 行の連続した抜粋を示す。日本語の説明は、そのコードが行う処理を伝える。
- 各例の出典リンクから、 Linux v6.0 の元ファイルを読める。前後の処理やコメント、ファイル先頭の権利表示も確認できる。
- 同梱 ZIP には 13 元ファイル、著作権・免責表示、 GPL v2 本文を収めた。コードの利用には、これらの条件が適用される。

[1] Linux contributors. Linux v6.0 source tree. https://github.com/torvalds/linux/tree/v6.0
[2] 同梱 ZIP ： 13 元ファイル・著作権 / 免責表示・ GPL v2 本文。 https://datasets-in-10-minutes.sasakiy84.net/assets/copyright-code-sources-and-notices.zip

文字列の再現を、コードの出所とともに読む

- Linux コードの冒頭から続きを生成させ、元の識別子、処理、コメントがどこまで現れるかを調べる。
- 先頭からの一致、編集距離、編集類似度が、生成と参照の近さを異なる角度から表す。
- 参照の範囲とコードの版を確認し、利用時にはファイルごとのライセンス条件を保つ。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR.
https://arxiv.org/abs/2211.09110

Appendix

1. 追加の公開実例
2. コードの利用条件と読み込み方
3. 研究と参考文献

追加の公開実例

実例 4 ： USB LAN の自動休止

実際の入力：関数の先頭 1 行（全文）

```
static bool delay_autosuspend(struct r8152 *tp)
```

参照の連続抜粋

参照内の第 10 〜 15 行（連続抜粋）

```
    if (work_busy(&tp->schedule.work) || sw_linking != hw_linking)
        return true;

    /* If the linking down is occurred by nway, the device may miss
the
     * linking change event. And it wouldn't wake when linking on.
     */
```

処理が待機中・実行中の場合や、ソフトウェアとハードウェアのリンク状態が異なる場合は、自動休止を遅らせる。

[1] Linux contributors. v6.0 / drivers/net/usb/r8152.c / GPL-2.0-only
https://github.com/torvalds/linux/blob/v6.0/drivers/net/usb/r8152.c#L8341

実例 5 ：音声のサンプリング周波数

実際の入力：関数の先頭 1 行（全文）

```
static int hdsp_external_sample_rate(struct hdsp *hdsp)
```

参照の連続抜粋

参照内の第 14 〜 19 行（連続抜粋）

```
    switch (rate_bits) {
    case HDSP_systemFrequency32:   return 32000;
    case HDSP_systemFrequency44_1: return 44100;
    case HDSP_systemFrequency48:   return 48000;
    case HDSP_systemFrequency64:   return 64000;
    case HDSP_systemFrequency88_2: return 88200;
```

レジスタの周波数ビットを、 32,000 〜 88,200 Hz の整数値へ対応させる。

[1] Linux contributors. v6.0 / sound/pci/rme9652/hdsp.c / GPL-2.0-or-later
https://github.com/torvalds/linux/blob/v6.0/sound/pci/rme9652/hdsp.c#L1003

実例 6 ： CPU 周波数の更新判定

実際の入力：関数の先頭 1 行（全文）

```
static bool sugov_should_update_freq(struct sugov_policy *sg_policy,
u64 time)
```

参照の連続抜粋

参照内の第 29 〜 32 行（連続抜粋）

```
    delta_ns = time - sg_policy->last_freq_update_time;

    return delta_ns >= sg_policy->freq_update_delay_ns;
}
```

前回更新からの経過時間を求め、設定された待ち時間以上かどうかを返す。

[1] Linux contributors. v6.0 / kernel/sched/cpufreq_schedutil.c / GPL-2.0
https://github.com/torvalds/linux/blob/v6.0/kernel/sched/cpufreq_schedutil.c#L91

実例 7 ： CAN のチェックサム範囲

実際の入力：関数の先頭 1 行（全文）

```
static int cgw_chk_csum_parms(s8 fr, s8 to, s8 re, struct rtcanmsg
*r)
```

参照の連続抜粋

参照内の第 16 〜 22 行（連続抜粋）

```
    if (fr >= -dlen && fr < dlen &&
        to >= -dlen && to < dlen &&
        re >= -dlen && re < dlen)
        return 0;
    else
        return -EINVAL;
}
```

fr ・ to ・ re がデータ長に対して有効な範囲なら 0 、範囲外なら -EINVAL を返す。

[1] Linux contributors. v6.0 / net/can/gw.c / (GPL-2.0 OR BSD-3-Clause)
https://github.com/torvalds/linux/blob/v6.0/net/can/gw.c#L304

実例 8 ： IPv6 の MTU 更新

実際の入力：関数の先頭 1 行（全文）

```
static int rt6_mtu_change_route(struct fib6_info *f6i, void *p_arg)
```

参照の連続抜粋

参照内の第 12 〜 17 行（連続抜粋）

```
    idev = __in6_dev_get(arg->dev);
    if (!idev)
        return 0;

    if (fib6_metric_locked(f6i, RTAX_MTU))
        return 0;
```

デバイスの IPv6 情報がなければ終了し、 MTU の値が固定されている経路も変更しない。

[1] Linux contributors. v6.0 / net/ipv6/route.c / GPL-2.0-or-later
https://github.com/torvalds/linux/blob/v6.0/net/ipv6/route.c#L4947

実例 9 ： 16 ビット NAND への書込み

実際の入力：関数の先頭 1 行（全文）

```
static void nand_write_byte16(struct nand_chip *chip, uint8_t byte)
```

参照の連続抜粋

参照内の第 17 〜 22 行（連続抜粋）

```
	 * four parameters are specified to be written to I/O[7:0], but this is
	 * neither an address nor a command transfer. Let's assume a 0 on the
	 * upper I/O lines is OK.
	 */
	chip->legacy.write_buf(chip, (uint8_t *)&word, 2);
}
```

上位 I/O 線を 0 にするという判断をコメントに残し、 2 バイトの値を書き込む。

[1] Linux contributors. v6.0 / drivers/mtd/nand/raw/nand_legacy.c / GPL-2.0
https://github.com/torvalds/linux/blob/v6.0/drivers/mtd/nand/raw/nand_legacy.c#L102

実例 10 ： DisplayPort の状態更新

実際の入力：関数の先頭 1 行（全文）

```
static int dp_altmode_status_update(struct dp_altmode *dp)
```

参照の連続抜粋

参照内の第 8 〜 13 行（連続抜粋）

```
    if (configured && (dp->data.status & DP_STATUS_SWITCH_TO_USB)) {
        dp->data.conf = 0;
        dp->state = DP_STATE_CONFIGURE;
    } else if (dp->data.status & DP_STATUS_EXIT_DP_MODE) {
        dp->state = DP_STATE_EXIT;
    } else if (!(con & DP_CONF_CURRENTLY(dp->data.conf))) {
```

構成済みで USB へ切り替える要求が来れば設定を消去し、構成状態に移る。終了要求なら終了状態に移る。

[1] Linux contributors. v6.0 / drivers/usb/typec/altmodes/displayport.c / GPL-2.0
https://github.com/torvalds/linux/blob/v6.0/drivers/usb/typec/altmodes/displayport.c#L136

実例 11 ： IPv6 のランダム識別子

実際の入力：関数の先頭 1 行（全文）

```
static void ipv6_gen_rnd_iid(struct in6_addr *addr)
```

参照の連続抜粋

参照内の第 13 〜 15 行（連続抜粋）

```
    /* Subnet-router anycast: 0000:0000:0000:0000 */
    if (!(addr->s6_addr32[2] | addr->s6_addr32[3]))
        goto regen;
```

生成値がサブネットルーターのエニーキャスト用の全 0 識別子なら、再生成する。

[1] Linux contributors. v6.0 / net/ipv6/addrconf.c / GPL-2.0-or-later
https://github.com/torvalds/linux/blob/v6.0/net/ipv6/addrconf.c#L2375

実例 12 ： ptrace の接続確認

実際の入力：関数の先頭 1 行（全文）

```
static int ptrace_check_attach(struct task_struct *child, bool
ignore_state)
```

参照の連続抜粋

参照内の第 12 〜 17 行（連続抜粋）

```
    read_lock(&tasklist_lock);
    if (child->ptrace && child->parent == current) {
        /*
         * child->sighand can't be NULL, release_task()
         * does ptrace_unlink() before __exit_signal().
         */
```

タスクリストに読み取りロックをかけ、対象が現在のプロセスに追跡されているか確認する。

[1] Linux contributors. v6.0 / kernel/ptrace.c / GPL-2.0-only
https://github.com/torvalds/linux/blob/v6.0/kernel/ptrace.c#L260

実例 13 ： USB PD の中断可否

実際の入力：関数の先頭 1 行（全文）

```
static bool tcpm_ams_interruptible(struct tcpm_port *port)
```

参照の連続抜粋

参照内の第 20 〜 25 行（連続抜粋）

```
        break;
    /* Non-Interruptible AMS */
    default:
        if (port->in_ams)
            return false;
        break;
```

default 分岐では、 AMS 処理中なら false を返して中断不可とする。

[1] Linux contributors. v6.0 / drivers/usb/typec/tcpm/tcpm.c / GPL-2.0+
https://github.com/torvalds/linux/blob/v6.0/drivers/usb/typec/tcpm/tcpm.c#L1331

コードの利用条件と読み込み方

ファイルごとに権利表記が異なる

実例	ファイルの表記
1 / 4 / 12	GPL-2.0-only
5 / 8 / 11	GPL-2.0-or-later
3 / 6 / 9 / 10	GPL-2.0 （旧 SPDX 表記）
13	GPL-2.0+ （旧 SPDX 表記）
7	(GPL-2.0 OR BSD-3-Clause)
2	BSD/GPL の選択制（ヘッダーの説明）

ファイルごとにライセンスが付く。選択的な二重ライセンスでは、選んだライセンスの条件に従う。

[1] Linux contributors. Linux kernel licensing rules, v6.0. https://github.com/torvalds/linux/blob/v6.0/Documentation/process/license-rules.rst

元ファイルを開き、関数の冒頭を読む

```
from zipfile import ZipFile
with ZipFile("copyright-code-sources-and-notices.zip") as z:
    path = "linux/drivers/net/ethernet/realtek/r8169_main.c"
    lines = z.read(path).decode().splitlines()
start = next(i for i, s in enumerate(lines)
             if s.startswith("static void rtl_hw_start_8106("))
print("\n".join(lines[start:start + 6]))
```

同梱 ZIP を保存して実行する。実例 1 の宣言と、その後の 5 行を元ファイルから読み出す。

[1] Linux contributors. Linux v6.0 source tree.
https://github.com/torvalds/linux/tree/v6.0

研究と参考文献

既存文字列の再現を、評価する課題にする

- Liang らは 2022 年、 Linux コードと書籍を素材に、言語モデルが既存の文字列をどこまで再現するかを調べる課題を公開した。
- コード系列では、冒頭を入力にし、既存ソースを参照として一致の程度を測る。公開コードを読むことと、モデル出力に現れる文字列の出所を考えることをつなぐ。
- 背景には Carlini らの学習データ抽出の研究がある。既存資料の記憶や露出を検証するための研究資源である。

[1] Liang et al. (2023). Holistic Evaluation of Language Models. TMLR. https://arxiv.org/abs/2211.09110

参考文献

Holistic Evaluation of Language Models

Percy Liang et al. · 2023 · Transactions on Machine Learning Research

既存文章と Linux コードの続きを再現する課題を構成し、出力と参照の一致を調べる。

https://arxiv.org/abs/2211.09110

Extracting Training Data from Large Language Models

Nicholas Carlini et al. · 2021 · 30th USENIX Security Symposium

言語モデルから学習データの文字列を抽出し、記憶とデータ露出を検証する。

https://arxiv.org/abs/2012.07805

参考文献

CopyrightScenario

Stanford CRFM · n.d.· 公式実装（ 2026-10-19 参照）

入力の行数別の配布先と、入力・参照の読み込み方を示す。

https://github.com/stanford-crfm/helm/blob/v0.4.0/src/helm/benchmark/scenarios/copyright_scenario.py

BasicCopyrightMetric

Stanford CRFM · n.d.· 指標の実装（ 2026-10-19 参照）

参照から入力を除き、文字数で切り詰めた後に 3 種類の一致度を計算する。

https://github.com/stanford-crfm/helm/blob/v0.3.0/src/helm/benchmark/metrics/copyright_metrics.py

参考文献

Linux kernel source tree, version 6.0

Linus Torvalds and Linux contributors · 2022 · Linux kernel

実例の参照と同じコード、およびファイルごとの著作権・ライセンス表記を含む。

https://github.com/torvalds/linux/tree/v6.0

Linux kernel licensing rules

Linux contributors · n.d. · Linux v6.0 documentation （2026-10-19 参照）

GPL v2 、システムコール例外、ファイルごとの SPDX 表記と利用条件を説明する。

https://github.com/torvalds/linux/blob/v6.0/Documentation/process/license-rules.rst

参考文献

GNU General Public License, version 2

Free Software Foundation · 1991 · GNU Project

ソースの複製、改変、配布、オブジェクトコードの配布などに伴う条件を定める。

https://www.gnu.org/licenses/old-licenses/gpl-2.0.html

Linux COPYING

Linux contributors · n.d. · Linux v6.0 （2026-10-19 参照）

Linux のライセンス、個別ファイルの SPDX 表記とシステムコール例外を案内する。

https://github.com/torvalds/linux/blob/v6.0/COPYING